

Il Linguaggio Rust: Breve Introduzione

Lezione 17

Sistemi Operativi - II Modulo
(Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

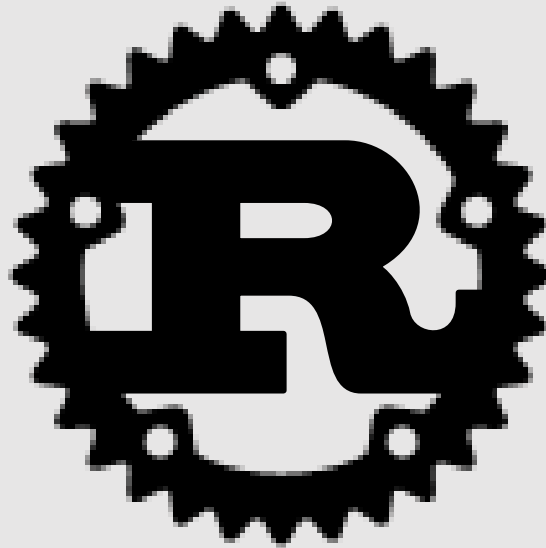


SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Errori di accesso alla memoria
- Breve storia di Rust
- Principali caratteristiche di Rust



Problema: Accesso corretto alla memoria

- Accessi 'illegali' alla memoria:
 - buffer overflows
 - use-after-free
 - de/referenziare puntatori nulli
 - accedere oltre la fine di un array
 - *etc.*
- 2019: circa il 70% delle vulnerabilità nel codice Microsoft è causato da accessi illegali alla memoria¹
- 2024/25: il governo USA scoraggia l'uso del C/C++^{2,3}

¹ <https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code>

² <https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf>

³ https://media.defense.gov/2025/Jun/23/2003742198/-1/-1/0/CSI_MEMORY_SAFE_LANGUAGES_REDUCING_VULNERABILITIES_IN_MODERN_SOFTWARE_DEVELOPMENT.PDF



La promessa di Rust

- Accessi illegali alla memoria: esempio di **comportamento non-definito**
- Dal man di `free()` : *“Any use of a pointer that refers to freed space results in undefined behavior.”*

- Rust **promette** che
 - Se il vostro codice compila senza errori, allora è **privo di comportamenti non-definiti**

Sicurezza della memoria in Rust

- Il compilatore Rust opera sul codice molti più controlli del C!
- Rust è un linguaggio fortemente e staticamente tipizzato
- Le garanzie di sicurezza di Rust sono basate su:
 - un **sistema di tipi basati sulla proprietà** di un «oggetto» (*ownership type system*)
 - un **controllo di tipo** (*type checking*) che assicura l'assenza di accessi illegali alla memoria
- Il controllo di tipo operato dal compilatore è una sofisticata analisi statica del codice
- **NON** c'è alcuna raccolta dei rifiuti (di memoria), o *garbage collection*!



Breve storia di Rust



- Sviluppo iniziato da Graydon Hoare (Mozilla) nel 2006
- Linguaggio per la programmazione di sistema: come il C, ma più sicuro!
- Rust 1.0 rilasciato nel maggio 2015 (versione attuale: 1.95, aprile 2026)
- Il compilatore Rust inizialmente scritto in OCaml, poi in Rust stesso (usando LLVM)
- Tutto il progetto Rust è codice aperto (open source)

Breve storia di Rust

- Rust è usato:

- in Mozilla Firefox

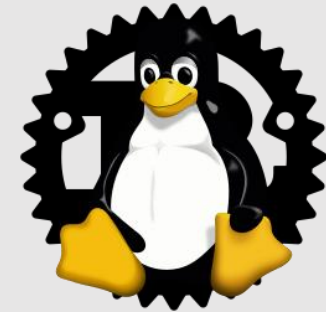


- in alcune librerie critiche di Windows



- nel kernel di Linux (driver, *etc.*); usabile al pari del C per lo sviluppo del kernel!

- ...



- Per installazione e documentazione:

<https://rust-lang.org/>

Principali caratteristiche di Rust

- Rust «mescola» diversi stili di programmazione:
 - Funzionale
 - Imperativa
 - Orientata agli oggetti
- La sintassi di Rust è di tipo **imperativo**, simile a C/C++
- Un ambiente di sviluppo completo:
 - `cargo` (gestore della compilazione, dipendenze, etc.)
 - `rustc` (il compilatore vero e proprio)
 - `rustdoc` (sistema per la documentazione automatica)

Esempio di programma Rust

- Vediamo un semplice programma Rust che calcola il MCD (massimo comun divisore) di una **lista** di numeri interi
- I numeri sono passati sulla linea di comando

Una funzione per il MCD di due interi

Una volta inizializzate, le 'variabili' **NON** possono essere modificate! Per farlo, dobbiamo aggiungere la clausola `mut` (*mutable*).

```
4 fn gcd mut n: u64, mut m: u64 -> u64 {  
5     assert!(n != 0 && m != 0);  
6     while m != 0 {  
7         if m < n {  
8             let t = m;  
9             m = n;  
10            n = t;  
11        }  
12        m = m % n;  
13    }  
14    n  
15 }
```

Tipo del **valore di ritorno** (intero senza segno 64bit).

Una **asserzione**. Se non è soddisfatta, il programma termina con un messaggio di errore. (E' una macro.)

Una **variabile locale**. (E' immutabile e il tipo è inferito da Rust!)

Valore di ritorno della funzione.



Unit test

- In Rust è possibile scrivere unit test **direttamente** nel codice
- Le unit test si **eseguono** con `cargo test`

E' un **attributo**. Indica che `test_gcd` è una **funzione di test**: non viene normalmente compilata. E' compilata ed eseguita quando si invoca `cargo test`.

```
17  #[test]
18  fn test_gcd() {
19      assert_eq!(gcd(14, 15), 1);
20
21      assert_eq!(gcd(2 * 3 * 5 * 11 * 17,
22                  3 * 7 * 11 * 13 * 19),
23                  3 * 11);
24  }
```

Assertione. Le due espressioni passate come parametro devono ritornare valori uguali.

La funzione `main()` del programma (I)

```
use std::str::FromStr;
use std::env;

fn main() {
    let mut numbers = Vec::new();

    for arg in env::args().skip(1) {
        numbers.push(u64::from_str(&arg).expect("error parsing argument"));
    }

    if numbers.len() == 0 {
        eprintln!("Usage: gcd NUMBER ...");
        std::process::exit(1);
    }

    let mut d = numbers[0];
    for m in &numbers[1..] {
        d = gcd(d, *m);
    }

    println!("The greatest common divisor of {:?} is {}", numbers, d);
}
```

Dichiarazioni di visibilità.

`FromStr` è un **trait** (insieme di metodi che un tipo può implementare).

Un vettore (prolungabile).

Ciclo per trattare la linea di comando; `env::args()` ritorna un **iteratore**.

Converte una stringa in `u64` e lo carica nel vettore. Gestione degli errori nella conversione. (Rust **no** ha le eccezioni.)

`d` conterrà il MCD (all'inizio è il primo elemento del vettore).

La funzione `main()` del programma (II)

```
let mut d = numbers[0];
for m in &numbers[1..]{
    d = gcd(d, *m);
}

println!("The greatest common divisor of {:?} is {}", numbers, d);
}
```

Idea principale: calcola l'MCD finale iterando su coppie di numeri (partendo dal secondo)

Come nel C, `&` ritorna l'indirizzo di una variabile

ATTENZIONE: `m` prende solo in prestito (*borrow*) l'indirizzo (o riferimento) degli elementi del vettore. La **proprietà** (*ownership*) del vettore rimane a `numbers`!

Come nel C, `*m` ritorna il valore puntato da `m`

Tipi di proprietà

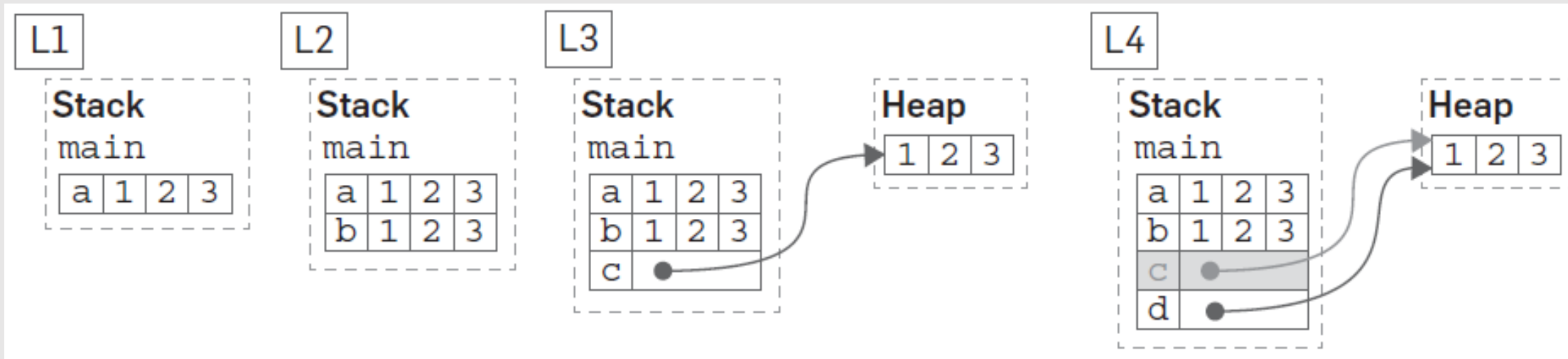
- Rust gestisce la memoria con un sistema di controlli (detto di **proprietà**) a tempo di compilazione
- Idea fondamentale:
 - Durante l'esecuzione di un programma, ad ogni istante di tempo le allocazioni heap sono di proprietà di una sola variabile
 - La memoria è liberata quando il proprietario non è più visibile, a meno che il proprietario non assegni la proprietà ad un'altra variabile

Tipi di proprietà

```
1 fn main() {  
2   let a = [1, 2, 3];    // L1  
3   let b = a;            // L2  
4   let c = Box::new(a);  // L3  
5   let d = c;            // L4  
6 }
```

Un semplice array nell'heap:
il proprietario è c.

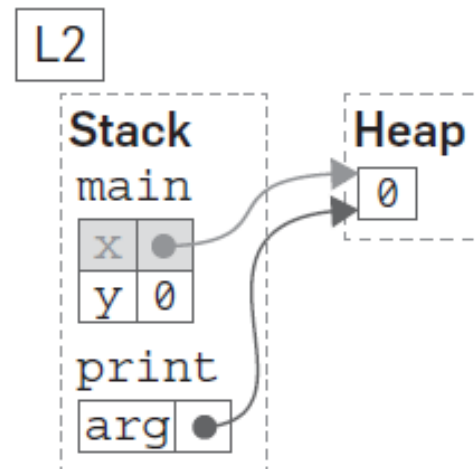
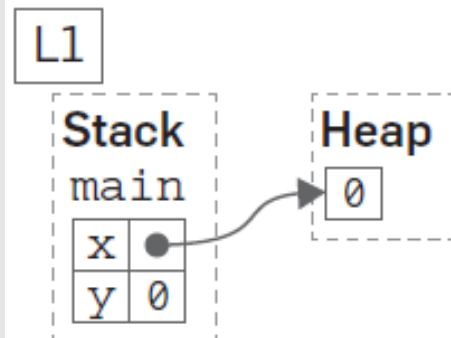
La proprietà dell'array passa a d!



Un esempio di prevenzione di errori

```
1 fn print(arg: Box<i32>) {  
2   println!("{}", arg); // L2  
3 }  
4  
5 fn main() {  
6   let x = Box::new(0);  
7   let y = *x; // L1  
8   print(x); // L3  
9   let z = *x; // L4  
10 }
```

Parametro passato per **riferimento**: la proprietà dell'argomento passa ad `arg`! Argomento deallocato al termine della funzione!



undefined behavior:
pointer used after
its pointee is freed

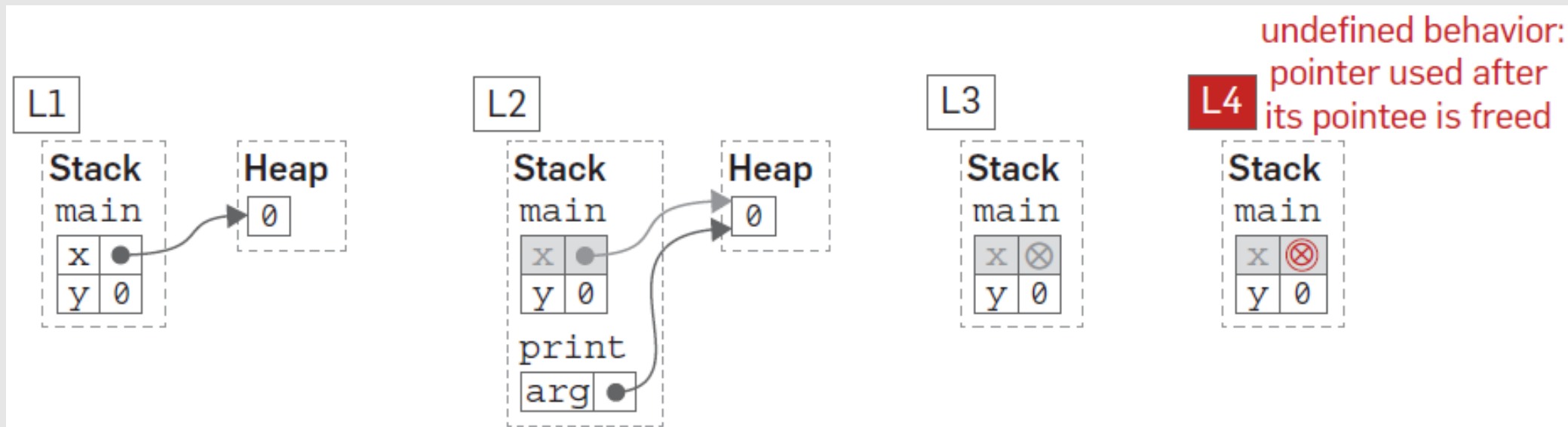


Un esempio di prevenzione di errori

```
1 fn print(Box<i32>) {  
2   print(); // L2  
3 }  
4  
5 fn main() {  
6   let x = ...; // L3  
7   let y = ...; // L4  
8   print(x);  
9   let ...  
10 }
```

Parametro passato per **riferimento**: la proprietà dell'argomento passa ad `arg`! Argomento deallocato al termine della funzione!

Rust rigetta il codice di cui sopra.



Prestare memoria

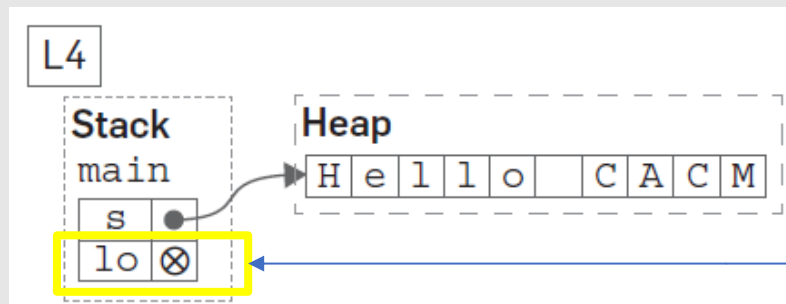
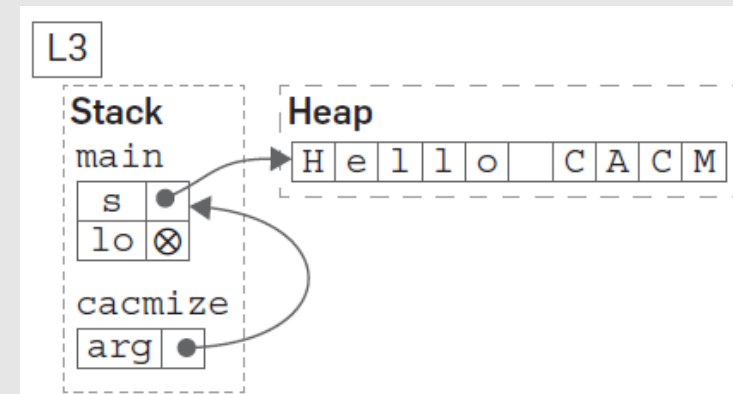
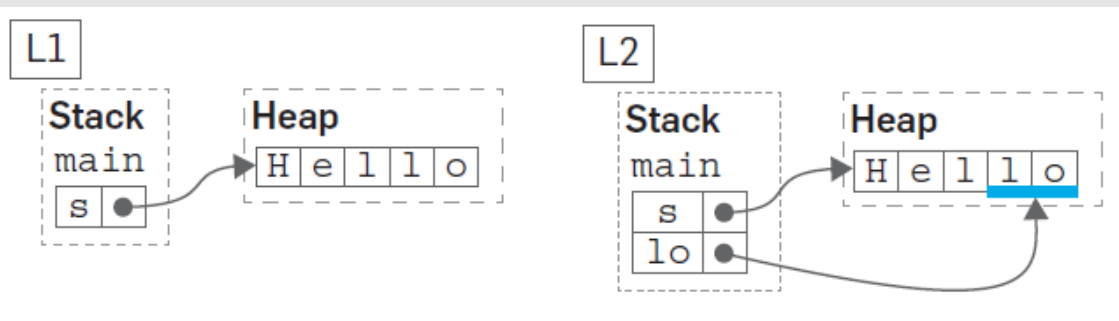
- Rust consente l'accesso a memoria heap senza averne la proprietà attraverso il meccanismo dei **riferimenti**
- Permettono di **prendere in prestito** memoria di proprietà di un'altra variabile
- Due tipi:
 - **immodificabile** (creato con `&`)
 - **modificabile** (creato con `&mut`)

Riferimenti

```
1 fn cacmize(arg: &mut String) {  
2     arg.push_str(" CACM"); // L3  
3 }  
4  
5 fn main() {  
6     let mut s = String::from("Hello"); // L1  
7     let lo = &s[3..5]; // L2  
8     cacmize(&mut s); // L4  
9 }
```

Riferimento modificabile

Riferimento non modificabile



lo non è più valido

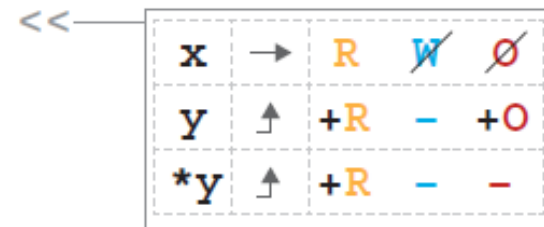
Prestare memoria

- I riferimenti operano con un sistema di **permessi**:
 - Lettura (R)
 - Scrittura (W)
 - Proprietà (O)

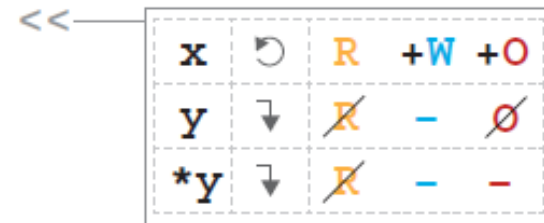
```
let mut x = String::from("Hello");
```



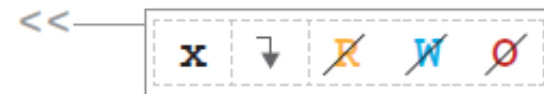
```
let y = &x;
```



```
println!("{}", x, y);
```



```
x.push_str(" world!");
```



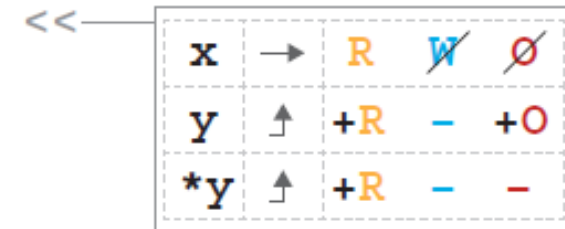
Prestare memoria

- Il controllo dei prestiti (*borrow checker*) di Rust assicura che le operazioni abbiano i permessi necessari
- Ad esempio:
 - Calcolare la lunghezza di un vettore richiede R
 - Concatenare una stringa con un'altra richiede W
 - Deallocare un vettore richiede O

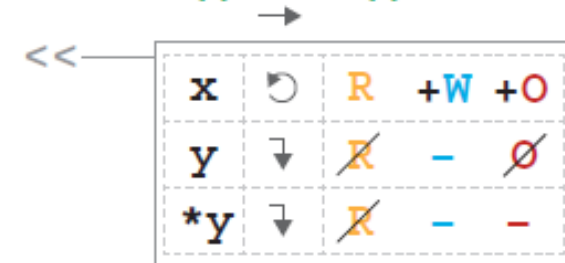
```
let mut x = String::from("Hello");
```



```
let y = &x;
```



```
println!("{}", x, y);
```



```
xRW.push_str(" world!");
```

```
println!("{}", x, y);
```

Prestare memoria

- Il controllo dei prestiti (*borrow checker*) di Rust assicura che le operazioni abbiano i permessi necessari
- Ad esempio:
 - Calcolare la lunghezza di un vettore richiede R
 - Concatenare una stringa con un'altra richiede W
 - Deallocare un vettore richiede O

```
let mut x = String::from("Hello");  
let y = ...  
println!("{}", x);  
x.push_str(" world!");  
println!("{}", x, y);
```

Domande?

